Reliable Delivery

為什麼這件事很難

在分散式系統裡，故障不是例外，而是常態。

伺服器會崩潰、網路會丟包、資料庫會變慢、依賴的第三方服務會抖動。你的服務正常運行時，它的每一個外部依賴，像是資料庫、快取、訊息佇列、下游服務，都有各自獨立的故障概率。把這些概率乘在一起，加上每天要處理的幾百萬個請求，故障在統計上就是必然發生的事。

真正的問題不是「如何防止故障發生」，而是「故障發生時，你的系統如何正確地應對」。

這篇講義把可靠資料傳遞拆解成六個相互關聯的概念：超時、重試、冪等性、退避加抖動、故障切換、降級回應。它們不是孤立的技術，而是一套相互依存的防線，每一個都建立在前一個的基礎上。理解它們如何協同運作，才是真正的系統設計能力。

超時（Timeout）

超時是最基本的防線，也是最常被遺忘的那個。

當你的服務向另一個服務發出請求，如果沒有設定超時，你就是在假設對方一定會在某個合理的時間內回應。但如果對方卡住了呢？你的連線就這樣懸著，不會釋放。如果這種情況大量發生，你的執行緒池（thread pool）被耗盡，連線池（connection pool）塞滿，你自己的服務也因此停止回應，一個依賴服務的故障，透過資源耗盡的機制，把你的服務也一起拉下去。這就是級聯故障（cascading failure）的標準路徑。

超時強迫你的系統在等待了足夠長的時間後，主動放棄並做出決定，而不是無限期地等下去。

超時的類型

理解不同類型的超時，才能正確地設定它們：

連線超時（Connection Timeout）：建立連線的最長等待時間。如果在這個時間內 TCP 握手還沒完成，就放棄。這通常應該設得相對短，比如幾百毫秒到幾秒，因為建立連線本身不應該花很長時間。

讀取超時（Read Timeout）：連線建立後，等待對方回傳資料的最長時間。這需要根據依賴服務的實際 SLA 來設定，不能太短（否則會誤殺正常請求），也不能太長（否則失去了超時的意義）。

bukanoat.org

寫入超時（Write Timeout）：把資料發送給對方的最長時間。在網路條件差或對方接收緩慢時可能觸發。

整體請求超時（Overall Request Timeout）：有時候你需要對整個請求設定一個端到端的預算，確保即使重試多次，整體等待時間也在可接受的範圍內。

```
import httpx

# 分別設定連線超時和讀取超時
timeout = httpx.Timeout(
    connect=1.0,   # 1 秒連線超時
    read=5.0,      # 5 秒讀取超時
    write=3.0,     # 3 秒寫入超時
    pool=2.0       # 2 秒等待連線池釋放連線
)

response = httpx.get("<https://api.example.com/data>", time
out=timeout)
```

怎麼設定超時值

超時值沒有放諸四海皆準的答案，但有一個思考框架：

參考依賴方的 SLA：如果下游服務的 P99 延遲是 200ms，設 2 秒的讀取超時是合理的緩衝。設 100ms 會引入太多的誤殺（false positive）；設 30 秒則讓超時幾乎沒有意義。

考慮整體鏈路的時間預算：如果你的 API 對用戶的承諾是 200ms 回應，那你給下游依賴的超時不能超過這個數字，還要留出自己的處理時間。

在面試中的說法：「我會對這個外部支付 API 設定 3 秒的讀取超時。正常的交易處理應該在 1 秒以內完成，3 秒提供了足夠的緩衝，同時確保一個卡住的依賴不會讓我們的執行緒無限期佔用。」

重試（Retry）

超時發生或者請求直接失敗時，第一個直覺是：再試一次。重試是處理暫時性故障（transient failure）最直接的手段。網路抖動導致的丟包、資料庫因為瞬間的高負載而超時、依賴服務因為垃圾回收而暫時沒回應，這些故障通常是短暫的，稍等片刻再試往往就會成功。

但重試不是萬靈丹，用錯了反而會讓情況更糟。

buildmoat.org

什麼時候應該重試

應該重試的情況：

- **網路暫時錯誤**：連線超時、讀取超時、`ECONNRESET` 這類網路層錯誤
- **5xx 伺服器錯誤**（有條件地）：`503 Service Unavailable`、`502 Bad Gateway` 通常代表暫時的過載，值得重試
- **資料庫連線失敗**：連線池暫時耗盡，稍後通常會有空閒連線

不應該重試的情況：

- **4xx 客戶端錯誤**：`400 Bad Request`、`401 Unauthorized`、`403 Forbidden`、`404 Not Found`，這些代表的是請求本身有問題，重試只是把同樣的錯誤請求又發一次，不會有任何幫助
- **業務邏輯錯誤**：「庫存不足」、「帳號餘額不夠」，這不是暫時的故障，而是系統正確運作的回應
- **非冪等操作（在還沒處理冪等性之前）**：重試一個建立訂單的請求，可能會建立兩筆一模一樣的訂單

最大重試次數

重試次數需要有上限。無限重試加上所有客戶端都在重試，等於把已經過載的系統推向更深淵。通常 3 到 5 次是合理的上限，超過了就應該讓請求失敗，讓上層的降級邏輯接管。

```
import time

def call_with_retry(fn, max_retries=3, retryable_status=(50
3, 502, 429)):
    for attempt in range(max_retries + 1):
        try:
            response = fn()
            if response.status_code in retryable_status:
                if attempt < max_retries:
                    continue  # 觸發重試
            return response
        except (ConnectionError, TimeoutError):
            if attempt == max_retries:
                raise  # 已達最大重試次數，讓錯誤往上傳
```

buildnoat.org

重試要和退避（backoff）搭配使用。不加退避的重試，等於在一個已經過載的服務上繼續倒垃圾。

冪等性（Idempotency）

在我們談退避之前，需要先解決重試帶來的最核心問題：**如果我重試了一個操作，我有沒有可能重複做了同一件事？**

這就是冪等性（idempotency）的意義：**同一個操作執行多次，和只執行一次，產生完全相同的結果**。

對讀取操作（GET）來說，冪等性是天然的，你查詢了三次用戶資料，資料庫裡的資料沒有任何改變，你拿到的結果也都一樣。問題出在寫入操作上。

天然冪等的操作

GET 是天然冪等的：讀取不改變狀態。

PUT（完整替換）是冪等的：用相同的資料 PUT 一個資源兩次，第二次 PUT 不會改變任何事情，資源已經是你想要的狀態了。

DELETE 是冪等的：刪除一個已經不存在的資源，和刪除一個存在的資源，最終結果相同，資源不存在。（第二次可能回傳 404，但系統狀態不變。）

POST（建立新資源）通常不是冪等的：建立兩次訂單，就會有兩筆訂單。這是最需要特別處理的情況。

冪等鍵（Idempotency Key）

讓非冪等操作變得冪等的標準解法是引入**冪等鍵**。客戶端在第一次發送請求時，附上一個唯一識別這次操作的 ID；如果需要重試，帶上相同的 ID。伺服器在處理請求之前，先查看有沒有已經處理過這個冪等鍵的記錄：有的話直接回傳之前的結果，沒有的話才執行操作並存下結果。

```
# 客戶端：產生並帶上冪等鍵
import uuid

idempotency_key = str(uuid.uuid4())  # 每次操作產生一次，重試時
複用同一個

response = requests.post(
    "/payments",
    json={"amount": 100, "user_id": "u123"},
```

buildmoat.org

```
    headers={"Idempotency-Key": idempotency_key}
)
```

```
# 伺服器端：處理冪等鍵
def create_payment(request):
    key = request.headers.get("Idempotency-Key")

    if key:
        # 查看這個 key 是否已經處理過
        cached = idempotency_store.get(key)
        if cached:
            return cached  # 直接回傳之前的結果，不重複執行

    # 執行實際操作
    result = process_payment(request.data)

    if key:
        # 存下結果，設定 TTL（例如 24 小時）
        idempotency_store.set(key, result, ttl=86400)

    return result
```

這個模式在支付系統中幾乎是必備的。Stripe 的 API 就是以冪等鍵為核心設計的，每次建立收費時，客戶端提供一個冪等鍵，Stripe 保證相同的鍵不會被收費兩次。

冪等性和訊息佇列：在非同步架構裡，訊息消費者（consumer）也需要是冪等的。因為「至少一次送達（at-least-once delivery）」是訊息佇列的常見保證，同一則訊息可能被消費多次。Consumer 必須能夠安全地處理重複的訊息，而不引發副作用。

退避加抖動（Backoff with Jitter）

不加退避的重試，是一個常見的、會讓問題指數級惡化的模式。

想像你的服務調用了一個外部 API，這個 API 因為過載開始返回 503。你的 100 個客戶端瞬間都失敗了，然後它們全部在一毫秒後重試。API 又收到了 100 個請求，又都失敗了。100 個客戶端又瞬間重試……這個循環會把一個已經岌岌可危的服務徹底壓垮，而且根本沒有機會恢復。這就是**驚群效應（thundering herd）**。

退避（backoff）的作用是讓重試間隔隨著失敗次數增加，給系統喘息的時間。

buildmoat.org

指數退避（Exponential Backoff）

最常用的退避策略。每次重試後等待的時間加倍：

```
第 1 次重試：等待 1 秒
第 2 次重試：等待 2 秒
第 3 次重試：等待 4 秒
第 4 次重試：等待 8 秒
第 5 次重試：等待 16 秒（通常會設一個上限，例如 30 秒）
```

```
import time

def retry_with_exponential_backoff(fn, max_retries=5, base_
delay=1.0, max_delay=30.0):
    for attempt in range(max_retries):
        try:
            return fn()
        except RetryableError:
            if attempt == max_retries - 1:
                raise

            delay = min(base_delay * (2 ** attempt), max_de
lay)
            time.sleep(delay)
```

指數退避比固定等待好很多，但還不夠。問題是所有客戶端的退避曲線是相同的，它們都在第 2 秒、第 4 秒、第 8 秒同時重試。雖然比立即重試好，但還是會形成同步的「脈衝」打在已經過載的服務上。

加上抖動（Jitter）

抖動（jitter） 是在退避等待時間上加入隨機性，讓不同客戶端的重試時間錯開。這把同步的脈衝打散成平滑的流量，讓依賴服務有機會逐步恢復。

```
import time
import random

def retry_with_jitter(fn, max_retries=5, base_delay=1.0, ma
x_delay=30.0):
```

buildmoat.org

```
    for attempt in range(max_retries):
        try:
            return fn()
        except RetryableError:
            if attempt == max_retries - 1:
                raise

            # Full Jitter：在 0 到計算出的退避時間之間隨機選
            cap = min(base_delay * (2 ** attempt), max_dela
y)
            delay = random.uniform(0, cap)
            time.sleep(delay)
```

AWS 的技術部落格把這種做法稱為「Full Jitter」，是實踐中最常被推薦的策略。另一種叫「Decorrelated Jitter」的變體讓每次等待時間都基於上一次的等待時間，隨機性更強，效果通常更好，但實作稍複雜。

在面試中，說出「指數退避加抖動（exponential backoff with jitter）」這幾個字，加上說明它解決的是同步重試的驚群效應，就已經是非常完整的回答了。

故障切換（Failover）

超時、重試和退避處理的是暫時的故障。但如果一個節點、一台伺服器、或整個資料中心永久性地掛掉了呢？這時候需要的是**故障切換（failover）**，把流量從掛掉的節點切換到健康的節點上。

負載平衡器的健康檢查

故障切換的第一個機制發生在負載平衡器層。負載平衡器定期對每台後端伺服器發送**健康檢查（health check）**請求，確認它還活著並且能正常處理請求：

- **TCP 健康檢查**：嘗試建立 TCP 連線，能建立就算健康
- **HTTP 健康檢查**：發送 HTTP GET 到 `/health` 端點，回傳 200 就算健康（可以在裡面加入更豐富的邏輯，例如確認資料庫連線正常、記憶體不超標等）

如果一台伺服器連續幾次健康檢查都失敗，負載平衡器把它從輪換池中移除，停止向它發送新的請求。當它恢復健康後，再把它加回來。整個過程對客戶端透明，客戶端只知道在打一個負載平衡器的位址，不知道背後的拓撲變化。

```
# 典型的 Nginx 健康檢查設定
upstream backend {
```

buildmoat.org

```
    server server1.example.com:8080;
    server server2.example.com:8080;
    server server3.example.com:8080;

    # 連續失敗 3 次就標記為不健康，等 30 秒再重試
    server server4.example.com:8080 max_fails=3 fail_timeou
t=30s;
}
```

資料庫故障切換

資料庫的故障切換比無狀態服務複雜得多，因為資料本身有狀態。

最常見的架構是一個 **Primary（主節點）** 接受所有寫入，加上一個或多個 **Replica（副本節點）** 透過複製保持和 Primary 同步。當 Primary 掛掉時：

1. **偵測故障**：監控系統或複製協定發現 Primary 無法回應
2. **選舉新 Primary**：從 Replica 中選出資料最新的那個，提升為新的 Primary
3. **更新連線資訊**：應用程式（或連線代理如 ProxySQL、RDS Proxy）切換到新的 Primary
4. **原 Primary 恢復後**：重新加入叢集，但以 Replica 的身份（需要先追上期間的資料差距）

在這個切換過程中，有一個不可避免的**停機視窗（downtime window）**，從故障被偵測到新 Primary 選出為止。現代管理型資料庫服務（AWS RDS Multi-AZ、Google Cloud SQL）把這個時間壓縮到幾秒到幾十秒，比手動操作快得多。

同步 vs 非同步複製的影響：

- **同步複製**：Primary 確認寫入前，必須等 Replica 也確認。故障切換時沒有資料遺失，但每次寫入的延遲更高
- **非同步複製**：Primary 確認寫入後，才非同步複製給 Replica。故障切換時可能遺失尚未複製的寫入（稱為「複製延遲 replication lag」），但寫入延遲更低

主動-主動 vs 主動-被動

主動-被動（Active-Passive）：只有主節點在處理請求，被動節點純粹是備援，平時接收複製的資料但不服務流量。故障時切換到被動節點。這是實作最簡單、資料一致性最強的方案。

主動-主動（Active-Active）：兩個（或多個）節點同時服務流量，互相複製。優點是利用率高、切換時無縫；缺點是需要解決寫入衝突，複雜度大幅上升。適合讀多寫

buildnoat.org

少、或能接受最終一致性的場景。

降級回應（Fallback）

故障切換是「找一個健康的同類來替代」。降級回應（fallback）是「當那個能力暫時不可用時，用一個較簡陋但能用的替代方案撐過去」。

兩者的目的相同，都是讓系統在部分故障時仍然能提供服務，只是策略不同。

快取資料作為降級回應

最常見的降級策略：當資料庫無法回應時，回傳快取裡的舊資料。

```
def get_product(product_id: str) -> dict:
    try:
        # 先嘗試從資料庫取最新資料
        product = db.query(f"SELECT * FROM products WHERE i
d = {product_id}")
        cache.set(f"product:{product_id}", product, ttl=30
0)
        return product
    except DatabaseError:
        # 資料庫掛了，嘗試從快取取舊資料
        cached = cache.get(f"product:{product_id}")
        if cached:
            return cached  # 回傳可能稍舊的資料，但總比錯誤頁好
        raise  # 快取也沒有就只能回傳錯誤了
```

這叫做「服役舊資料（serve stale data）」。在很多場景下，比如商品資訊頁、用戶個人頁面，稍舊一點的資料比完全無法訪問好得多。

預設值和靜態回應

當功能性的資料完全不可用時，回傳一個「說得過去的預設值」：

- **推薦系統掛了**：回傳預先計算好的「熱門商品」靜態列表，而不是個人化推薦
- **個人化設定讀取失敗**：使用系統預設的設定
- **評分統計服務超時**：顯示「評分暫時不可用」而不是整頁崩潰

熔斷器（Circuit Breaker）

buildmoat.org

降級回應需要一個機制來**觸發**它。這就是熔斷器（circuit breaker）的角色，它監控對下游依賴的呼叫，當錯誤率超過閾值時，自動停止把請求打到那個依賴，改而立刻觸發降級邏輯。

熔斷器有三個狀態：

```
Closed（正常）→ 失敗率超過閾值 → Open（熔斷）
                                      ↓ 等待一段時間
                                  Half-Open（半開）
                                  ↙          ↘
                              測試請求成功      測試請求失敗
                               → Closed         → Open
```

Closed（閉路）：正常狀態，所有請求都正常轉發給下游依賴。監控失敗率。

Open（開路）：熔斷狀態。所有請求不再打到下游，直接觸發降級回應（回傳快取、預設值或錯誤）。下游服務得到喘息機會恢復。

Half-Open（半開路）：熔斷一段時間後，放一個測試請求進來。成功了就恢復正常（回到 Closed）；失敗了就繼續熔斷（回到 Open）。

熔斷器解決了一個重要問題：如果沒有它，你的服務在下游掛掉期間仍然不斷地把請求打過去，每個請求都要等到超時才能失敗，這期間你的執行緒被大量佔用。熔斷器讓失敗「快速失敗（fail fast）」，立刻返回，不等待，釋放資源。

```
class CircuitBreaker:
    def __init__(self, failure_threshold=5, reset_timeout=3
0):
        self.failure_count = 0
        self.failure_threshold = failure_threshold
        self.state = "closed"
        self.reset_timeout = reset_timeout
        self.last_failure_time = None

    def call(self, fn, fallback):
        if self.state == "open":
            # 檢查是否到了嘗試半開路的時間
            if time.time() - self.last_failure_time > self.
reset_timeout:
                self.state = "half-open"
            else:
```

buildnat.org

```
                return fallback()  # 直接走降級路徑

        try:
            result = fn()
            if self.state == "half-open":
                self.reset()  # 成功了，恢復正常
            return result
        except Exception:
            self.record_failure()
            return fallback()

    def record_failure(self):
        self.failure_count += 1
        self.last_failure_time = time.time()
        if self.failure_count >= self.failure_threshold:
            self.state = "open"

    def reset(self):
        self.failure_count = 0
        self.state = "closed"
```

這些概念如何串在一起

這六個概念不是獨立的工具，而是一套協作的防線。理解它們的關係比記住每個定義更重要：

```
請求發出
    ↓
設定了超時 → 超時發生 → 觸發重試
                                ↓
                        加上退避（防驚群效應）
                                ↓
                        重試是安全的嗎？→ 需要冪等性保證
                                ↓
                        失敗率太高？→ 熔斷器觸發
                                ↓
                        走降級路徑（快取、預設值）
```

buildmoat.org

↓
底層基礎設施故障？→ 故障切換到健康節點

什麼時候在面試裡用這些

主動說明可靠性策略

不要等面試官問「如果這個服務掛掉怎麼辦」才開口。在設計外部依賴時，主動說明你的可靠性策略：

「這個地方我們調用了支付 API。我會設定 3 秒的讀取超時，對 5xx 錯誤做指數退避重試（最多 3 次，加上 jitter 防止驚群效應）。每個支付請求帶上冪等鍵，確保重試不會導致重複扣款。如果失敗率在 30 秒內超過 50%，熔斷器打開，直接回傳『支付服務暫時不可用』的錯誤，而不是讓用戶等到超時。」

這一段話就涵蓋了超時、重試、退避加抖動、冪等性、熔斷器，完整而自然。

常見面試情境

設計 Uber 叫車系統：「司機位置更新每秒可能幾百萬次，我用 Kafka 做緩衝。消費者（Consumer）處理位置更新是冪等的（重複處理同一個位置更新不會有副作用），所以用至少一次語義就夠了，不需要精確一次的額外複雜度。」

設計通知系統：「發送通知的任務進入訊息佇列。Consumer 在成功調用通知 API 後才提交 offset（至少一次）。我們在通知記錄表裡加上 `notification_id + status` 的唯一索引，讓 Consumer 在重複消費時能快速識別已發送的通知，不會重複推播。」

設計電商訂單系統：「建立訂單的 API 要求客戶端帶上冪等鍵（通常由前端在用戶點擊『下單』時生成，重試時複用相同的鍵）。訂單服務在 Redis 裡查詢這個 key，如果已存在就直接回傳之前建立的訂單 ID，避免網路重試導致的重複下單。」

資料庫高可用：「我們的 PostgreSQL 使用 Multi-AZ 部署，同步複製到一個 Standby 節點。當 Primary 故障時，AWS RDS 自動把 Standby 提升為新的 Primary，通常在 60 秒以內完成。應用程式連線到 RDS Proxy，由它管理連線切換，應用程式不需要感知故障切換的細節。」

常見的 Deep Dive 問題

「如果訊息被消費兩次，你的系統還能正確工作嗎？」

這是測試你是否真正理解冪等性的問題，不只是能說出定義。

回答要說出具體的實現方式：「在 Consumer 處理訊息的邏輯裡，我會先查詢一個去重表（deduplication table），用訊息的 `message_id` 做唯一鍵。如果這個 ID 已存在，

bytebytego.org

就跳過處理，直接提交 offset。如果不存在，處理訊息並在同一個事務裡寫入去重表。這樣即使同一則訊息被投遞兩次，第二次的處理會被冪等性機制攔截。」

「你怎麼選擇超時時間？」

不要說「看情況」。說出你的決策框架：

「我會先測量依賴服務的 P99 延遲，假設是 150ms。然後設超時為 P99 的 2 到 3 倍，例如 400ms。這提供了合理的緩衝而不讓異常的慢請求佔用太久的資源。

另外要考慮整體鏈路的時間預算。如果我們對用戶的 SLA 是 500ms，那這個依賴的超時必須遠小於 500ms，留出我們自己的處理時間和其他依賴的時間。超時值需要在真實流量下持續監控和調整，不是一次設好就不管了。」

「熔斷器打開後，後端服務恢復了，你怎麼知道？」

這是考察你對 Half-Open 狀態的理解：

「熔斷器在 Open 狀態等待一個固定的冷卻時間（例如 30 秒），然後進入 Half-Open 狀態。在 Half-Open 狀態下，允許一個探針請求打過去。成功了就認為服務恢復，把熔斷器切回 Closed；失敗了就重新進入 Open 並重置計時器。這個機制讓系統能在不需要人工介入的情況下自動偵測依賴的恢復。

需要注意的是：在 Half-Open 期間，如果探針請求成功，不要立刻放開所有流量，應該逐步放量（traffic ramp-up），防止突然的全流量把剛恢復的服務又打垮。」

總結

可靠的資料傳遞不是一個單一的技術決策，而是一套從細粒度到粗粒度、層層疊加的防線：

- **超時**：讓故障快速失敗，不讓它無限期地佔用資源
- **重試**：對暫時的故障給第二次機會，對永久的故障快速放棄
- **冪等性**：讓重試變得安全，確保同一個操作多次執行不會產生副作用
- **退避加抖動**：讓重試有節制，防止集體重試把已經脆弱的系統壓垮
- **故障切換**：在節點或資料庫故障時，把流量自動切換到健康的替代節點
- **降級回應**：當某個能力暫時不可用時，用一個較簡陋但能用的替代方案撐過去

面試中，這些概念最有價值的展示方式，不是逐個列出它們，而是在設計外部依賴的時候，自然地說出你的可靠性策略，表達你在畫架構圖的同時，也在思考每一條線上可能出現的問題，以及如何讓系統在那些問題發生時優雅地應對。

buildmoat.org